



Embedded Software for the Internet Of Things

**Programming for Efficient
Computing**

Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

Low-level Optimizations

- What should the compiler be able to do?
- What could stop the compiler from optimizing?
- How can we tell if the compiler has applied the optimizations?
- How can we modify the source code to help the compiler optimize?

Starting Points for Efficient Code

- Write code, check functionality, then optimize.

“We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil”

— Donald Knuth



The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming.

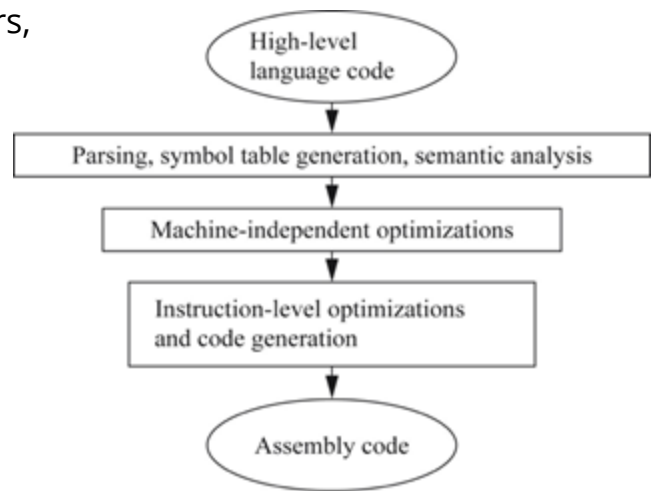
Starting Points for Efficient Code

- Use a top-down approach.
- Know your microprocessor's architecture, compiler, and programming language.
- Use the right tool for the problem
- Consider the use of optimized libraries for critical paths in your program.

What can a compiler do?

Review of Compiler Stages

- Parser
 - reads in source code e.g. C code, checks for lexical and syntax errors,
 - forms intermediate code (tree representation)
- High-Level Optimizer
 - Modifies intermediate code (processor-independent)
- Code Generator
 - Creates assembly code step-by-step from each node of the intermediate code, Allocates variable uses to registers
- Low-Level Optimizer
 - Modifies assembly code (parts are processor-specific)
- Assembler
 - Creates object code (machine code)
- Linker/Loader
 - Creates executable image from object file



What should the compiler be able to do?

- **Machine-Independent (MI)**

- Eliminate code with no effect
- Specialize computations
- Eliminate redundant computations

- **Machine-Dependent (MD)**

- Take advantage of special hardware features
- Manage or hide latency
- Manage limited machine resources

Models of Programs

- Source code is not a good representation for programs:
 - Clumsy, leaves much information implicit.
- Compilers derive **intermediate representations** to manipulate and optimize the program.
- Our fundamental model for programs is the **control/data flow graph (CDFG)**
 - Models both data operations and control operations.

Data Flow Graphs (DFG)

- A **data flow graph (DFG)** is a model of a program with **no conditionals**.
- In a high-level programming language, a code segment with no conditionals, with only one entry and exit point, known as a **basic block**.

`w = a + b;`

`x = a - c;`

`y = x + d;`

`x = a + c;`

`z = y + e;`

Data Flow Graphs (DFG)

- Two assignments to the variable x.
 - Rewrite the code in **single-assignment form** -> a variable appears only once on the left side.

```
w = a + b;
```

```
x = a - c;
```

```
y = x + d;
```

```
x = a + c;
```

```
z = y + e;
```



```
w = a + b;
```

```
x1 = a - c;
```

```
y = x1 + d;
```

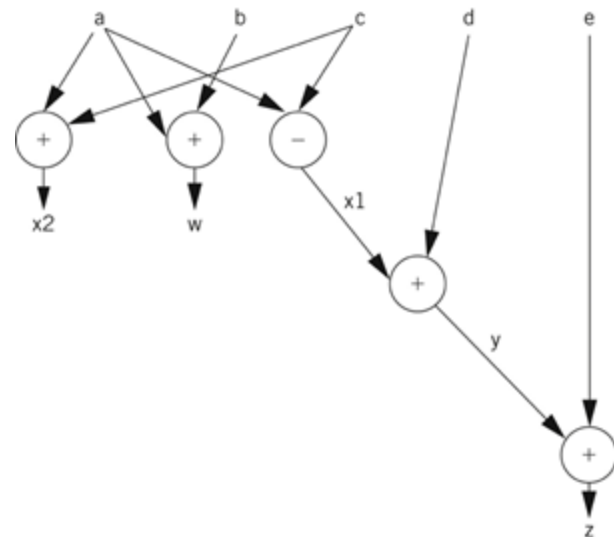
```
x2 = a + c;
```

```
z = y + e;
```

Data Flow Graphs (DFG)

- The single-assignment form means the data flow graph is **acyclic**:
 - If x is assigned multiple times, then the second assignment would form a cycle in the graph.
- Keeping the data flow graph acyclic is important in many types of analyses.

```
w = a + b;  
x1 = a - c;  
y = x1 + d;  
x2 = a + c;  
z = y + e;
```



Data Flow Graphs (DFG)

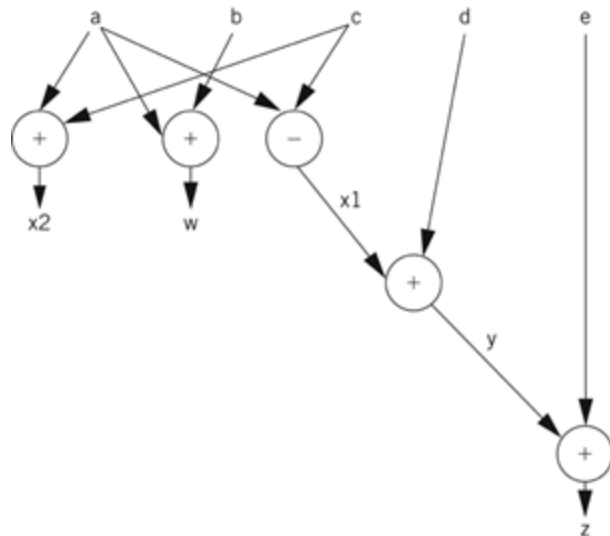
- The data flow graph for the code makes **the order** in which the operations are performed.
- We can use it to determine feasible **reorderings** of the operations with **no dependencies**:
 - E.g., to reduce pipeline or cache conflicts.

Partial order:

$a+c$, $a+b$, $a-c$; $x1+d$; $y+e$

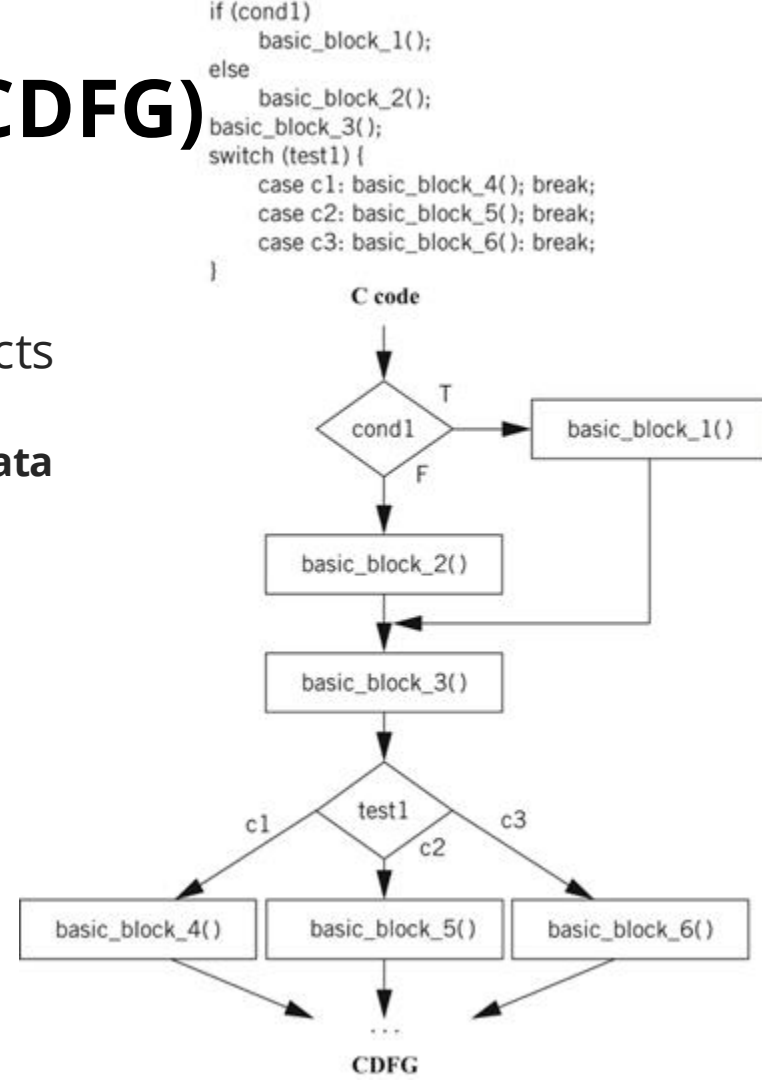
Can do pairs of operations **in any order**.

```
w = a + b;  
x1 = a - c;  
y = x1 + d;  
x2 = a + c;  
z = y + e;
```



Control-Data Flow Graph (CDFG)

- CDFG (Control-data flow graph) uses a data flow graph as an element and adds constructs **to describe control**.
 - has two types of nodes: **decision nodes** and **data flow nodes**.



Machine-Independent (MI) Optimizations

MI: Eliminate code with no effect

- **Useless code** (based on data-flow graph - DFG)
- Mark **critical operations** -> important ones that really **affect the results!**
 - For example, operations that
 - set return value for procedure -> what is the operand of this statement?
 - input/output statement
 - modifies non-local data -> direct effect on the results!
- Find operations which define **data (operands)** used by these critical operations
- Repeat until set of critical operations doesn't grow
- **Delete remaining operations**

MI: Eliminate code with no effect

```
#define DEBUG 0  
...  
if (DEBUG) dbg(p1);
```

- **Unreachable code**

- Examine CFG for nodes **without predecessors**

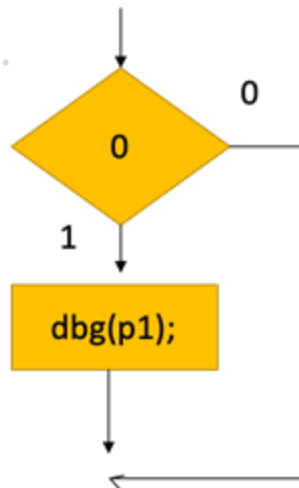
- Code that does not affect the program's observable behavior.

Initial Code:

```
int Dead(void){  
    int a=10;  
    int z=50;  
    int c;  
    c=z*5;  
    printf(c);  
    a=20;  
    a=a*10; //No need of These Two  
Lines  
    return 0;  
}
```

Optimized Code:

```
int Dead(void) {  
    int a=10;  
    int z=50;  
    int c;  
    c=z*5;  
    printf(c);  
    return 0;  
}
```



MI: Eliminate code with no effect

```
#define DEBUG 0  
...  
if (DEBUG) dbg(p1);
```

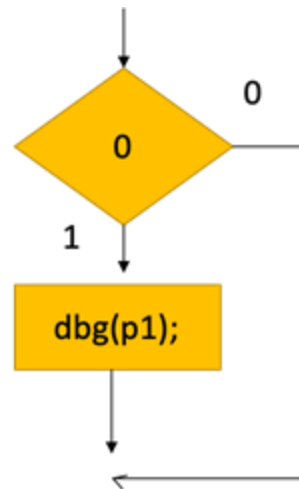
- **Unreachable code**
 - Examine CFG for nodes **without predecessors**
- Code that does not affect the program's **observable behavior**.

Initial Code:

```
int Dead(void) {  
    int a=10;  
    int z=50;  
    int c;  
    c=z*5;  
    printf(c);  
    a=20;  
    a=a*10; //No need of These Two Lines  
    return 0;  
}
```

Optimized Code:

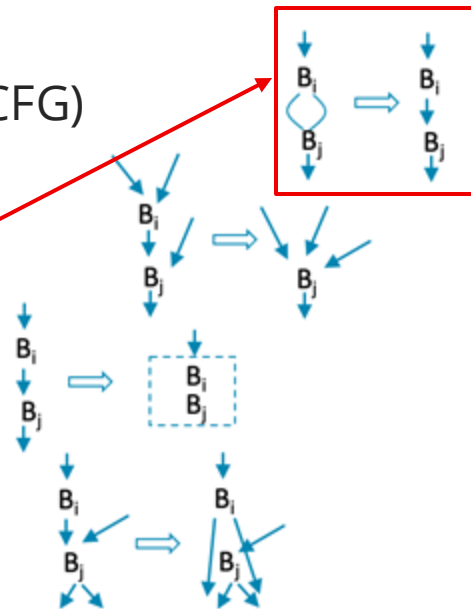
```
int Dead(void) {  
    int a=10;  
    int z=50;  
    int c;  
    c=z*5;  
    printf(c);  
    return 0;  
}
```



MI: Eliminate code with no effect

- **Useless control flow** (based on control flow graph - CFG)
 - Fold **redundant branches**: e.g., two branches into one

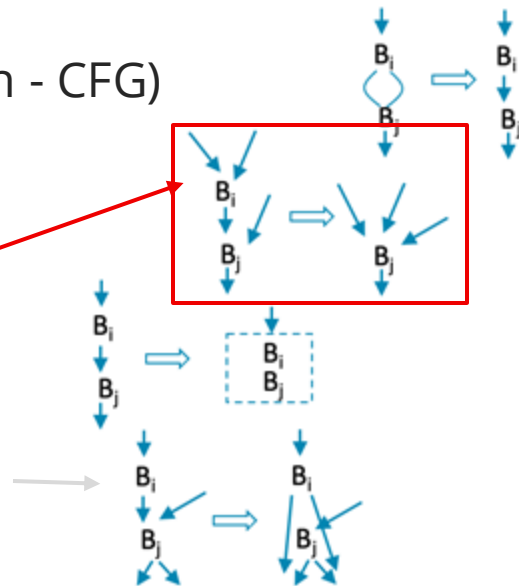
E.g., no need for *if/then/else*



MI: Eliminate code with no effect

- **Useless control flow** (based on control flow graph - CFG)
 - Remove **empty blocks**: B_i is empty, remove it

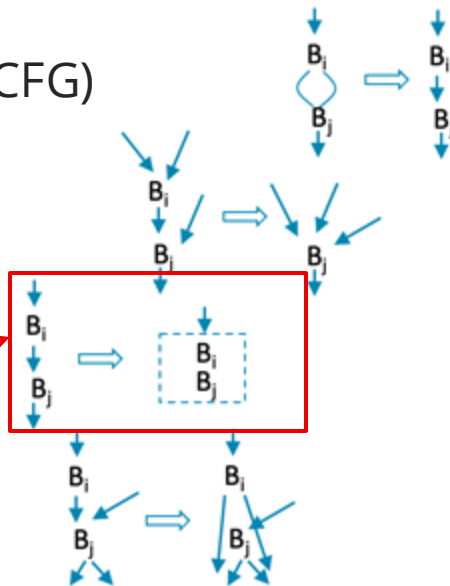
E.g., B_i has no affect on the result



MI: Eliminate code with no effect

- **Useless control flow** (based on control flow graph - CFG)
 - Combine blocks: **merge blocks**

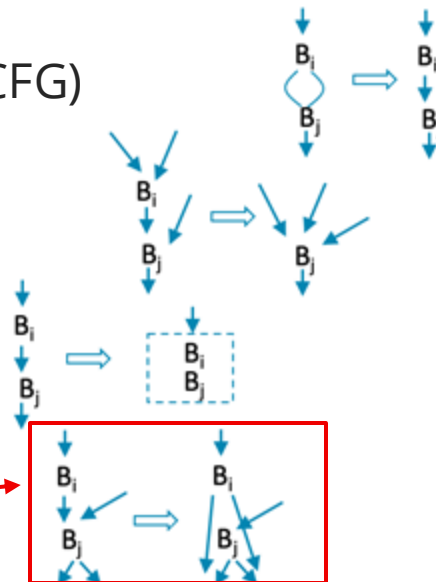
E.g., no need for a branch



MI: Eliminate code with no effect

- **Useless control flow** (based on control flow graph - CFG)
 - **Branch hoisting**: replace jump to empty block with jump to its **successor**;
 - e.g., as a result of previous optimization, B_j was reduced to an empty block $\rightarrow$ adjust jumps from B_i

E.g., remove jump from B_i to B_j by forwarding the jump to the successor of B_j



MI: Eliminate code with no effect

- **Simplification** of algebraic identities
 - $x * 1 = x, x + 0 = x$

MI: Specialize computations

- **Operator strength reduction** `y = x * 2;`  `y = x << 1;`
 - Replace multiplication with addition or shifting, division with subtraction or shifting
- **Constant propagation**
 - if a variable has a known value at a given point in the program, it might possible to specialize operations based on this knowledge
 - *e.g., perform calculations at compile time rather than runtime.*
- **Peephole optimization**
 - Recognize patterns of assembly instructions which can be replaced with a faster set

Examples: Strength Reduction

- The operators that consume higher execution time are replaced by the operators consuming less execution time.

Initial code:

```
y = x * 2;
```

Optimized code:

```
y = x + x;      or      y = x << 1;
```

Initial code:

```
y = x / 2;
```

Optimized code:

```
y = x >> 1;
```

Examples: Constant propagation

- Avoid additional computation.

Initial code:

```
x = 2 * 3;
```

Optimized code:

```
x = 6;
```

Examples: Redundant load and store elimination

- Redundancy is eliminated.

Initial code:

```
y = x + 5;  
i = y;  
z = i;  
w = z * 3;
```

Optimized code:

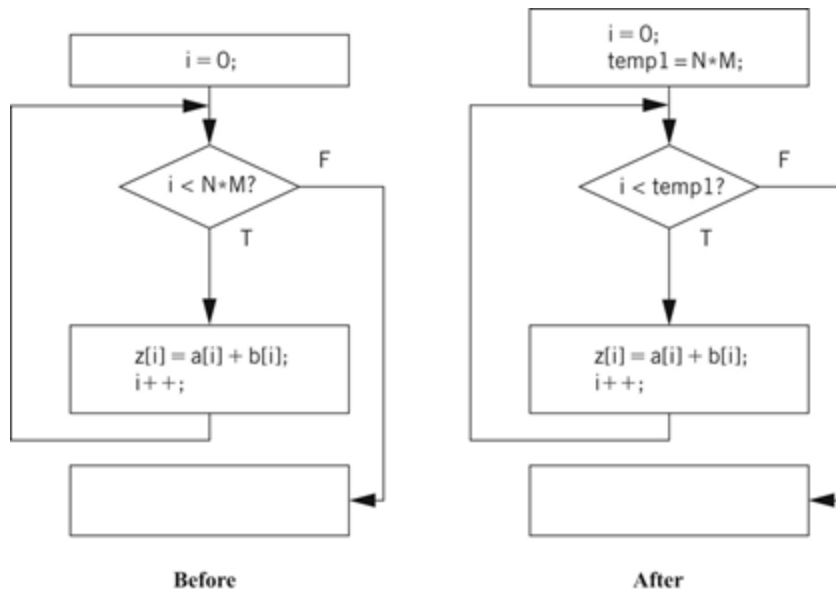
```
y = x + 5;  
w = y * 3; /* there is no i now
```

```
// We've removed two redundant variables i & z whose value were just  
being copied from one another.
```

MI: Code Motion

- Move unnecessary code out of a loop.
- If a result does not depend on operations performed in the loop body.

```
for (i=0; i<N*M; i++)  
  z[i] = a[i] + b[i];
```



MI: Induction Variable Elimination

- Induction variable
 - A variable whose value is derived from the loop iteration variable's value.
- The compiler often introduces induction variables to implement the loop.

```
for (i=0; i<N; i++)  
  for (j=0; j<M; j++)  
    z[i,j] = b[i,j];
```



```
for (i = 0; i<N; i++)  
  for (j = 0; j<M; j++) {  
    zbinduct = i*M + j;  
    *(zptr + zbinduct) = *(bptr + zbinduct);  
  }
```



```
zbinduct = 0;  
for (i = 0; i < N; i++) {  
  for (j = 0; j < M; j++) {  
    *(zptr + zbinduct) = *(bptr + zbinduct);  
    zbinduct++;  
  }  
}
```

MI: Enabling Transformations

- **Loop unrolling**

- replicate loop body
 - **Faster** by reducing loop overhead
 - At the expense of **increased** binary code size

```
for (i = 0; i<4; i++) {  
    a[i]=b[i]*c[i];  
}
```



```
for (i=0; i<2; i++) {  
    a[i*2] = b[i*2] * c[i*2];  
    a[i*2+1] = b[i*2+1] * c[i*2+1];  
}
```



```
a[0] = b[0]*c[0];  
a[1] = b[1]*c[1];  
a[2] = b[2]*c[2];  
a[3] = b[3]*c[3];
```

MI: Enabling Transformations

- Loop fusion
 - combines two or more loops into a single loop.
 - the loops must iterate over the same values.
 - the loop bodies must not have dependencies

```
for (i=0; i<N; i++)  
    a[i] = b[i] * 5;  
  
for (j=0; j<N; j++)  
    w[j] = c[j] * d[j];
```



```
for (i=0; i<N; i++) {  
    a[i] = b[i] * 5;  
    w[i] = c[i] * d[i];  
}
```

MI: Enabling Transformations

- **Loop unswitching**

- hoist loop-invariant control-flow operations out of the loop

```
for (i=0; i<N; i++) {  
    if(x>y)  
        a[i] = b[i] * x;  
    else  
        a[i] = b[i] * y;  
}
```



```
if(x>y){  
    for (i=0; i<N; i++)  
        a[i] = b[i] * x;  
} else {  
    for (i=0; i<N; i++)  
        a[i] = b[i] * y;  
}
```

MI: Enabling Transformations

- **Renaming**

- value numbering gives independent name to each value defined
- optimizer can recognize **already-computed values** which are **still live**

$a = x + y$

$b = x + y$

$a = 17$

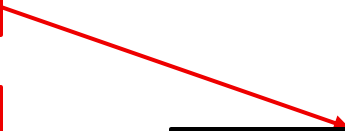
$c = x + y$

$a_0 = x_0 + y_0$

$b_0 = x_0 + y_0$

$a_1 = 17$

$c_0 = x_0 + y_0$



No need to
compute.

Machine-Dependent Optimizations

Machine-Dependent Optimizations

- Take advantage of **special instructions, addressing modes** and **hardware features**
 - Instruction pre-fetch, branch prediction (special processor circuitry)
 - Conditional instruction execution (Arm instructions)
 - Advanced addressing modes (processor ISA dependent)
- Manage or hide latency (pipeline, memory system, ALU)
 - Large memory latency – reschedule operations to keep the processor busy
 - Rearrange loop iteration order for **cache locality** (e.g. to use GPU local memory)
 - Change data layout to improve **cache locality**
- Manage limited machine resources
 - **Register allocation**

**What could stop the compiler
from optimizing?**

Excessive Variable Scope

- **Avoid** declaring variables as **globals** or **statics** when they could be locals
 - Globals and statics are allocated permanent storage in memory, not reusable stack space
- Compiler assumes that any function may access a global variable
 - This might create side effects -> be careful when access to globals
 - *Function must write back any globals it has modified before calling a subroutine*
 - *Function must reload any globals to use after calling the routine*

Floating Points

- MSP432 (ARM) has hardware support for floating point
 - VMUL (floating-point) Floating-point multiply.
 - VLDR (floating-point)
 - Extension register load.

```
3
4 /**
5  * main.c
6  */
7 void main(void)
8 {
9     WDT_A->CTL = WDT_A_CTL_PW | WDT_A_
10
11     float x = 1.2f;
12
13     x = x*0.45;
14
15     printf("%f",x);
16
17     return;
18 }
19
```

```
..... **
9     WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD;
00002a96: 490F          ldr        r1, [pc, #0x3c]
00002a98: F44F40B5     mov.w     r0, #0x5a80
00002a9c: 8008          strh     r0, [r1]
11     float x = 1.2f;
00002a9e: 480B          ldr        r0, [pc, #0x2c]
00002aa0: 9000          str        r0, [r13]
13     x = x*0.45;
00002aa2: 480B          ldr        r0, [pc, #0x2c]
00002aa4: EE000A90     vmov      s1, r0
00002aa8: ED9D0A00     vldr      s0, [r13]
00002aac: EE200A80     vmul.f32  s0, s1, s0
00002ab0: ED8D0A00     vstr      s0, [r13]
15     printf("%f",x);
00002ab4: 9800          ldr        r0, [r13]
00002ab6: F000F84B     bl        aeabi_f2d
```

Automatic Promotions in Arithmetic Expressions

- How are expressions with mixed data types evaluated?

```
float f;  
char c;  
int r;  
  
r = f * c;
```

Automatic Promotions in Arithmetic Expressions

```
float f;  
char c;  
int r;  
  
r = f * c;
```

- How are expressions with mixed data types evaluated?
- ANSI C Standard has rules for arithmetic conversions
 - If either operand is a long double, promote the other to a long double
 - Else if either is a double, promote the other to a double
 - Else if either is a float, promote the other to a float
 - Else if either is a unsigned long int, promote the other to a unsigned long int
 - Else if either is a long int, promote the other to a long int
 - Else if either is an unsigned int, promote the other to an unsigned int
 - Else both are promoted to int: short, char, bit field
- *special rules for dealing with signed/unsigned differences left out*

Resulting Object Code

```
float f;  
char c;  
int r;  
  
r = f * c;
```

- Call routine to convert c to float
- Call routine to perform floating point multiply with f
- Call routine to convert result from floating point to integer
- store result in r

- Time and code space overhead of conversion routines
- Avoid mixed type expressions

In our case... (MSP432)

- VCVT (between floating-point and integer)
 - Convert between floating-point numbers and integers.

```
1 #include "msp.h"
2
3
4 /**
5  * main.c
6  */
7 void main(void)
8 {
9     WDT_A->CTL = WDT_A_CTL_PW | WDT_A_
10     float f = 1.2f;
11     char c = 8;
12     int r = 65534;
13
14     r = f * c;
15
16     printf("%d", r);
17
18     return;
19 }
```

```
10
00002866: 480F      ldr      r0, [pc, #0x3c]
00002868: 9000      str      r0, [r13]
11
char c = 8;
0000286a: 2008      movs     r0, #8
0000286c: F88D0008  strb.w   r0, [r13, #8]
12
int r = 65534;
00002870: F64F70FE  movw     r0, #0xfffe
00002874: 9001      str      r0, [r13, #4]
14
r = f * c;
00002876: ED9D0A00  vldr     s0, [r13]
0000287a: F89D0008  ldrb.w   r0, [r13, #8]
0000287e: EE000A90  vmov     s1, r0
00002882: EEF80A60  vcvf.f32.u32 s1, s1
00002886: EE200A80  vmul.f32 s0, s1, s0
0000288a: EEBD0AC0  vcvf.s32.f32 s0, s0
0000288e: EE100A10  vmov     r0, s0
00002892: 9001      str      r0, [r13, #4]
16
printf("%d", r);
```

ANSI C Standard for Argument Promotions

- Function arguments smaller than an int for **non-prototyped functions** are promoted to ints
 - Extra time converting to int
 - Extra space on stack

```
int Find_Average(char a, char b, char c, char d) {  
    . . .  
}
```

ANSI C Standard for Argument Promotions

- Function arguments smaller than an int for **non-prototyped functions** are promoted to ints
 - Extra time converting to int
 - Extra space on stack
- So **prototype all functions!**
 - If program is broken into modules, put prototype in header (.h) file
 - Otherwise put prototype near top of C code file, before the function is called

```
int Find_Average(char a, char b, char c, char d);  
. . .  
int Find_Average(char a, char b, char c, char d) {  
. . .  
}
```

Volatile Keyword

- The declaration of a variable as volatile tells the compiler that the variable can be modified at any time externally to the implementation, for example:
 - By the operating system.
 - By another thread of execution such as an interrupt routine or signal handler.
 - By hardware.
- This declaration ensures that the compiler **does not optimize** any use of the variable on the assumption that this variable is unused or unmodified.

Volatile Keyword

- **Potential problems** when not using volatile
 - When a volatile variable is not declared as volatile, the compiler assumes that its value cannot be modified from outside the scope that it is defined in.
 - Therefore, the compiler might perform **unwanted optimizations**. This problem can manifest itself in various ways:
 - *Code might become stuck in a loop while polling hardware*
 - *Multi-threaded code might exhibit strange behavior.*
 - *Optimization might result in the removal of code that implements deliberate timing delays.*

Volatile Keyword

- Assume that **buffer_full** is a variable that can be changed by an ISR or a device can directly modify it.
 - This means the state of **buffer_full** can change asynchronously with program flow.

```
int buffer_full;
int read_stream(void) {
    int count = 0;
    while (!buffer_full)
    {
        count++;
    }
    return count;
}
```

Volatile Keyword

- Because **buffer_full** is not declared as volatile, the compiler assumes that its value cannot be modified **outside** the program.
 - Having already read the value of buffer_full into r0
 - the compiler omits reloading the variable when optimizations are enabled,
 - because its value cannot change.
 - The result is the **infinite loop** labeled .LBB0_1.

```
int buffer_full;  
int read_stream(void) {  
    int count = 0;  
    while (!buffer_full)  
    {  
        count++;  
    }  
    return count;  
}
```



```
read_stream:  
    movw    r0, :lower16:buffer_full  
    movt    r0, :upper16:buffer_full  
    ldr     r1, [r0]  
    mvn     r0, #0  
.LBB0_1:  
    add     r0, r0, #1  
    cmp     r1, #0  
    beq     .LBB0_1      ; infinite loop  
    bx     lr
```

Volatile Keyword

- Now, the compiler assumes that the value of `buffer_full` can change outside the program and performs **no optimization**.
 - Therefore, the value of `buffer_full` is loaded into register `r2` inside the loop labeled `.LBB1_1`.
 - As a result, the assembly code that is generated for loop `.LBB1_1` is correct.

```
volatile int buffer_full;  
int read_stream(void) {  
    int count = 0;  
    while (!buffer_full)  
    {  
        count++;  
    }  
    return count;  
}
```



```
read_stream:  
    movw    r1, :lower16:buffer_full  
    mvn     r0, #0  
    movt    r1, :upper16:buffer_full  
.LBB1_1:  
    ldr     r2, [r1]      ; buffer_full  
    add     r0, r0, #1  
    cmp     r2, #0  
    beq     .LBB1_1  
    bx      lr
```

Compiler Optimization Levels e.g. Arm Compiler

- **Optimizing for code vs. speed**

- **-Otime:** the compiler aggressively optimizes for time e.g. aggressive inlining, at the expense of a possible increase in image (binary) size.
- **-Ospace:** the compiler optimizes for image size at the expense of a possible increase in execution time.

- **Optimization levels:**

- **-O0 :** Minimum optimization. Turns off most optimizations. Best possible debug view.
- **-O1 :** Restricted optimization e.g. removes unused inline functions. Turns off optimizations that seriously degrade the debug view.
- **-O2 :** High optimization. e.g. compiler automatically inlines functions if optimizing for time. The compiler may perform optimizations that cannot be described by debug information.
- **-O3 :** Maximum optimization e.g. loop unrolling if optimizing for time. This can give significant performance benefits at a small code size cost, but at the risk of a longer build time.

How can we tell if the compiler has applied the optimizations?

Profiling Computer Programs

- Code profilers are tools that **instrument** computer programs
 - either the program source code or its binary form(s)
- Code profilers can measure the frequency and execution time of certain instructions or function calls, memory accesses and memory usage, event trace etc.
- Code profilers are crucial for code optimization e.g. for speed, memory footprint, energy/power.
- Some of the most popular profilers include:
 - Statistical profilers such as Shark (OSX), oprofile (Linux), VTune (Intel) and Streamline (Arm).
 - Intermediate language instrumentation tools such as OpenPAT.
 - Runtime instrumentation tools such as **Valgrind** and DynamoRIO.

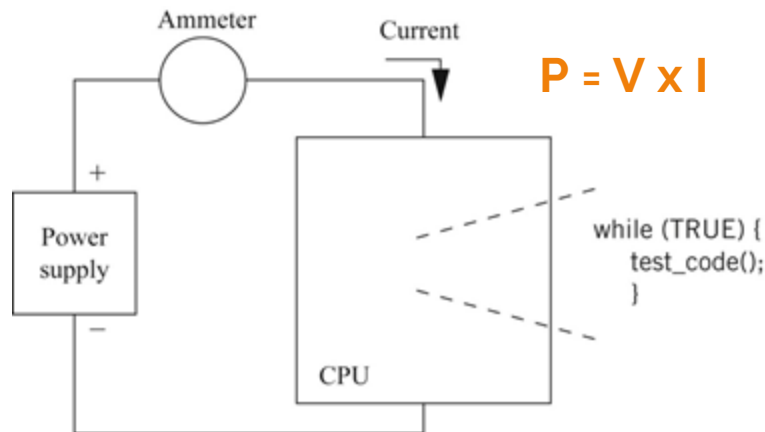
How can we modify the source code to help the compiler optimize?

Iterative Optimization Process

- Read the Compiler Manual 
- Apply the necessary hints/directives for the compiler to achieve the sought optimizations
- Inspect the results
- Go back to 1 and/or 2 if the results are not satisfactory

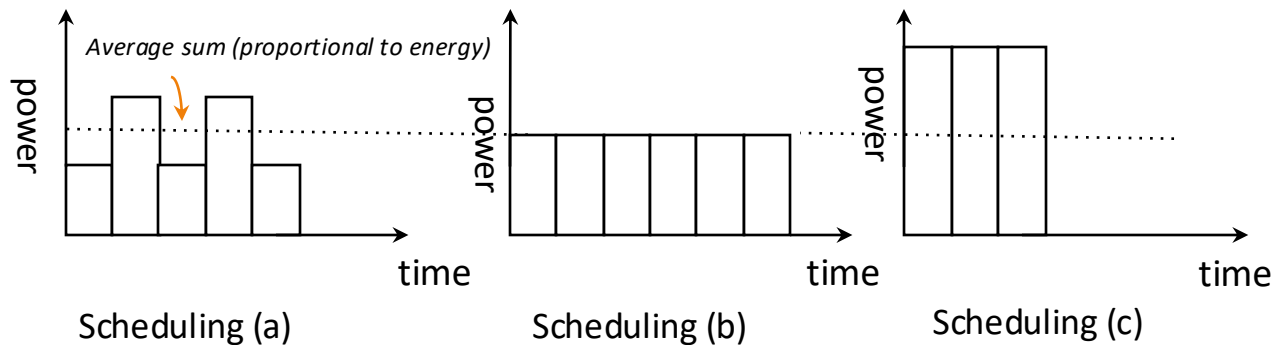
Optimizing For Power and Energy

- **Optimizing for power** aims to reduce the amount of energy consumed over a small interval of time
- **Optimizing for energy** aims to optimize the total energy consumed.
 - Power is linked to heat, energy is the total you pay for in the end.



Different Power but Same Energy.

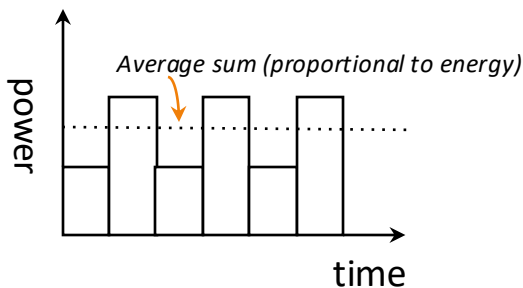
- Assume the power consumption is defined by the **number of instructions** executed at any moment in time.
 - Uniform power consumption across all instructions.
- By rescheduling instructions in a parallel computer system, we can have different power consumption profiles with the same total energy



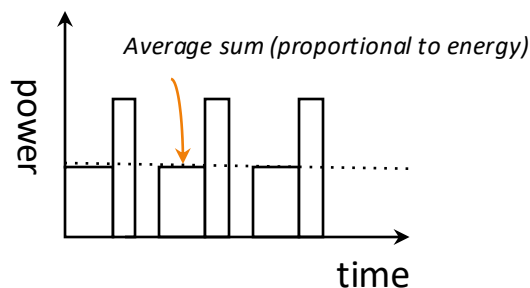
Power consumption changes with the parallelization.

Lower Energy but Different Power Budget.

- Suppose we can execute certain instructions faster (e.g. in half the time) by using certain **hardware features**, e.g. DSPs offering hardware multiplication.

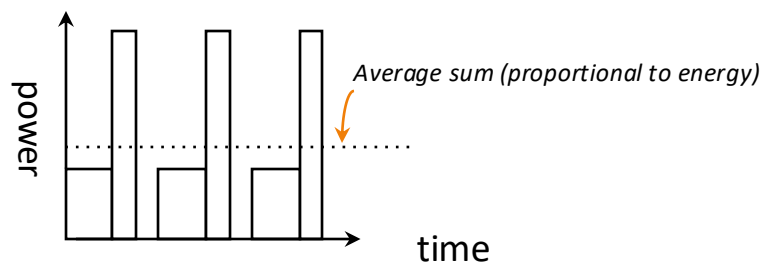


Scheduling (a)



Scheduling (d) exploits certain hardware features (using the same power budget)

Lower energy/same
power budget



Scheduling (e) exploits certain hardware features (at a higher power budget this time)

Lower energy/**higher**
maximum power

Is Optimizing For Power/Energy The Same As Optimizing For Code Size?

- Reducing code size results in less memory usage which can reduce power/energy consumption especially that **memory access is often power/energy hungry**.
- Eliminating redundant variables for instance reduces code size and power/energy consumption.
- But reducing code size might also be achieved at the expense of **more computations**, which consume extra power/energy.
 - Any example?

Is Optimizing For Power/Energy The Same As Optimizing For Code Size?

```
temp= x;  
x=y;  
y=temp;
```

```
x= x + y;  
y= x - y;  
x= x - y;
```

- Only two variables are used in the code to the right (instead of three in the code to the left).
 - Which code is better? Assume MOV is better than ADD and SUB.

High Level Optimizations

Low Power Computing

- There are many ways in which we can reduce power consumption:
 - Reduce the amount of computations necessary
 - *e.g. optimize programs to remove redundant operations, use optimized libraries, reduce memory transactions by reusing local data, **use better algorithms**.*
 - Toolchain optimization
 - *e.g. help the compiler do a better job through directives to exploit hardware-specific features.*
 - Exploit power efficient hardware-assisted techniques
 - *e.g. Dynamic Voltage and Frequency Scaling (DVFS).*
 - Improve use of hardware resources
 - *e.g. better scheduling algorithms*

Optimization Risks

- Unpredictability of development effort needed
 - Balancing act
 - Pro: expected performance gain
 - Cons: additional development time required
- Difficulties in prediction
 - How much gain will we get after this optimization?
 - *Will it be optimized enough so we can stop optimizing the program?*
 - How long will it take to perform this optimization?
 - How many more optimizations will we need?

Optimization Risks

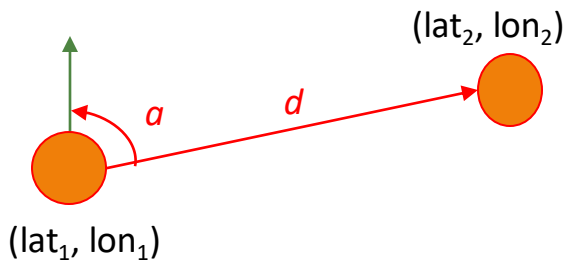
- Impact on code maintainability
 - Code will be used in future
 - *Bug fixes, feature changes, feature additions, upgrades*
 - What if you've forgotten how your optimized code works?
 - What if someone else needs to maintain your optimized code?
 - Optimization often **hurts code maintainability**
 - Need to optimize in a way which retains maintainability

High Level Optimizations: Do Less Work

- Fundamental concept: **perform less computation**
 - Lazy (or deferred) execution: **don't compute data until needed**
 - Early decisions: for decisions based on computations, may be able to use intermediate results
- Applied broadly
 - Many algorithms implement these concepts
 - Compilers try to apply these in optimization passes
- Role of developer
 - Implement concepts directly in source code
 - Help the compiler apply these concepts

Example Program: “Nearby Points of Interest”

- Find distance from current position to the closest position of a fixed set of positions.
 - Positions are described as coordinates on the surface of the Earth (latitude, longitude)



$$d = \text{acos}((\sin(lat_1) * \sin(lat_2) + (\cos(lat_1) * \cos(lat_2) * \cos(lon_2 - lon_1))) * 6371$$

$$a = \text{atan2}(\cos(lat_1) * \sin(lat_2) - \sin(lat_1) * \cos(lat_2) * \cos(lon_2 - lon_1), \sin(lon_2 - lon_1) * \cos(lat_2)) * \frac{180}{\pi}$$

Example Program: "Nearby Points of Interest"

- **Calc_Distance** is called on every point, returning **closest_d** (in km)

```
float Calc_Distance( PT_T *p1,  const PT_T *p2) {  
    // calculates distance in kilometers between locations  
    return acos(sin(p1->Lat*PI/180) * sin(p2->Lat*PI/180) +  
                cos(p1->Lat*PI/180) * cos(p2->Lat*PI/180) *  
                cos(p2->Lon*PI/180 - p1->Lon*PI/180)) * 6371;  
}
```



$d = \text{acos}((\sin(\text{lat}_1) * \sin(\text{lat}_2) + (\cos(\text{lat}_1) * \cos(\text{lat}_2) * \cos(\text{lon}_2 - \text{lon}_1))) * 6371$

```
void Find_Nearest_Point( . . . ) {  
    . . .  
    while (strcmp(points[i].Name, "END")) {  
        d = Calc_Distance (&ref, &(points[i])  
    );  
        if (d < closest_d) {  
            closest_d = d;  
            closest_i = i;  
        }  
        i++;  
    }  
}
```

Example Program: “Nearby Points of Interest”

```
float Calc_Distance( PT_T * p1,  const PT_T * p2) {  
    // calculates distance in kilometers between locations  
    return acos(sin(p1->Lat*PI/180)* sin(p2->Lat*PI/180) +  
                cos(p1->Lat*PI/180)*cos(p2->Lat*PI/180)*  
                cos(p2->Lon*PI/180 - p1->Lon*PI/180)) * 6371;  
}
```

- Distance is **acos(big_expression)*6371**
 - What is 6371?
 - *Scaling factor to convert angle in radians to distance in km*
 - 6371km = Earth's circumference/ 2π radians
- Can compare angle (returned by acos) rather than distance
 - Angle is still **proportional to** distance between points

Example Program: “Nearby Points of Interest”

- Optimized Code
 - Eliminates **NPoints-1** floating point multiplies
 - PT_T structure keeps precalculated sin and cos expressions.

```
float Calc_Distance_in_Radians( PT_T *p1,  const PT_T* p2)
{
    // calculates distance in radians between locations
    return acos(p1->SinLat * p2->SinLat +
        p1->CosLat * p2->CosLat * cos(p2->Lon - p1->Lon));
    // no *6371 here
}

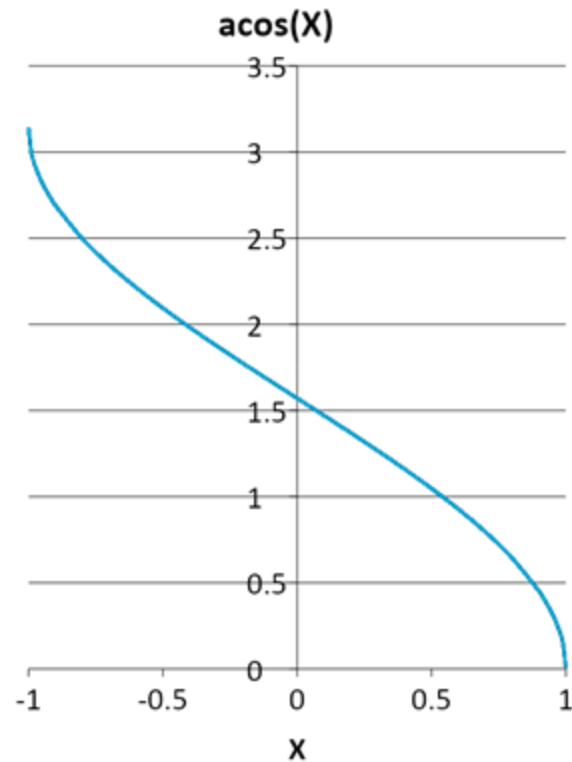
void Find_Nearest_Point( . . . ) {
    while (strcmp(points[i].Name, "END")) {
        d = Calc_Distance_in_Radians(&ref, &(points[i])) ;
        . . .
    }
    *distance = d*6371;
    . . .
}
```

p1->SinLat

```
float Calc_Distance( PT_T *p1,  const PT_T *p2) {
    // calculates distance in kilometers between locations
    return acos(sin(p1->Lat*PI/180) * sin(p2->Lat*PI/180) +
        cos(p1->Lat*PI/180)*cos(p2->Lat*PI/180)*
        cos(p2->Lon*PI/180 - p1->Lon*PI/180)) * 6371;
}
```

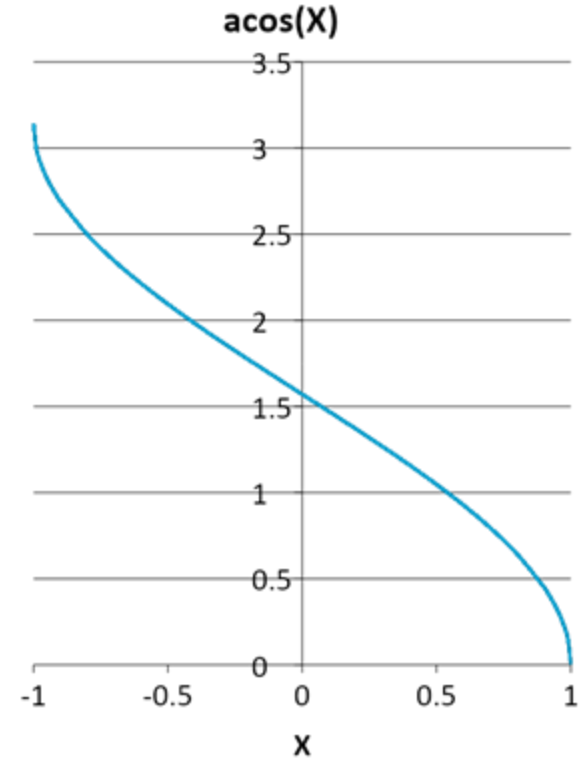
Example Program: “Nearby Points of Interest”

- Can we make distance comparisons without using `acos`?
 - How is `acos` related to its argument X ?
 - Can we call `acos` **just once** – to compute the distance to the closest point?



Example Program: “Nearby Points of Interest”

- acos always **decreases** when input X increases
- Nearest point will have minimum distance and **maximum X**
- So search for point with maximum argument to acos function
 - After finding nearest point (max X), compute $\text{distance_km} = \text{acos}(X) * 6371$



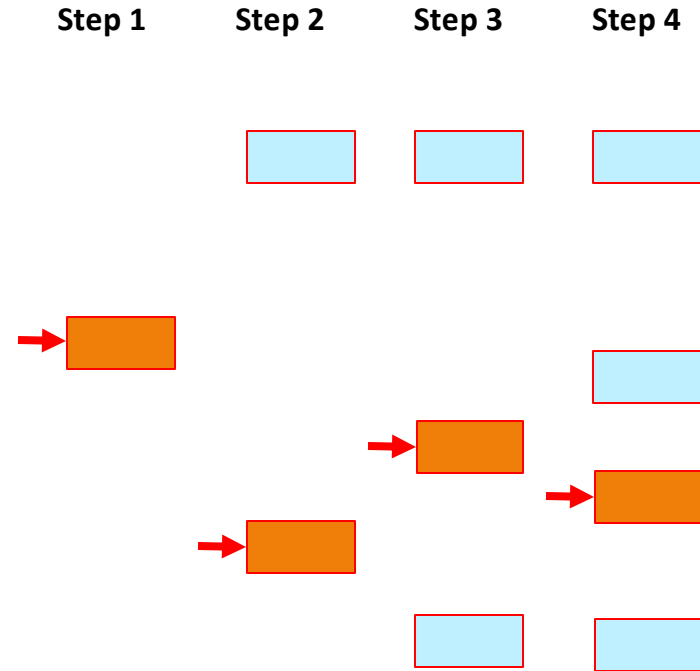
Example Program: “Nearby Points of Interest”

- Optimized Code
 - Eliminates **NPoints-1** floating point acos calculations.

```
float Calc_Distance_inverse ( PT_T * p1,  const PT_T * p2) {  
    // calculates distance in radians between locations  
    return (p1->SinLat * p2->SinLat +  
            p1->CosLat * p2->CosLat * cos(p2->Lon - p1->Lon); // no acos here  
}  
  
void Find_Nearest_Point( . . . ) {  
    while (strcmp(points[i].Name, "END")) {  
        d = Calc_Distance_inverse(&ref, &(points[i]) );  
        if (d>closest_d) closest_d = d;  
        i++;  
    }  
    *distance = acos(d)*6371;  
    . . .  
}
```

Use Better Algorithms: E.g., Binary Search

- Divide and Conquer approach
 - Regions in table **must be sorted** by increasing starting address
 - For each entry, start address $\leq$ end address
- The time complexity is reduced to $O(\log(n))$ compared to $O(n)$ for a brute force search (n is the number of elements).



Memory Access Pattern is Very Important

- Relative energy per operation:
 - **memory transfer: 33**
 - **external I/O: 10**
 - **SRAM write: 9**
 - **SRAM read: 4.4**
 - **multiply: 3.6**
 - **add: 1**

Bringing data from
memory/IO to CPU is
very expensive!

Better Memory Access Pattern

- Memory transfers are the most **expensive** operation
 - a memory transfer: tens or hundreds of times more energy than an arithmetic operation.
- Poorly organizing instructions and data in memory might lead to a lot of memory transfers.
 - the biggest payoffs in energy optimization!
- **Accesses to registers** are the most energy efficient
- Cache accesses are more energy efficient than main memory accesses.

Use of Optimized Libraries

- The use of optimized libraries can achieve your goals at a relatively small extra investment.
 - The use of optimized libraries however can make your code **less portable** across different platforms.
- Some of the most widely used optimized libraries are **Math libraries**
 - **NAG (Numerical Algorithm Group) Numerical Library:** Software library of numerical analysis routines e.g. linear algebra, optimization, differential equations, and regression analysis.
 - **Intel Math Kernel Library (MKL):** optimized Maths routines (e.g. linear algebra, sparse solvers, FFTs) for science, engineering, and financial applications. Optimized specially for Intel processors.
 - **CMSIS-DSP:** Software library with a suite of common signal processing functions (e.g. filters, transforms, matrix algebra) for use on energy-efficient Cortex-M processor based devices.

Precision

- Match the data types and approximations method to the range and accuracy needs of your algorithm.
 - You can **trade-off** accuracy/precision for speed, code size, energy/power.
- Floating point arithmetic is needed when you are dealing with a large range of data values that fixed-point arithmetic cannot deal with.
 - Single precision floating point uses 32 bits (8 bits for exponent, 24 for fraction mantissa)
 - Double-precision floating point precision uses 64 bits (11 bits for exponent, 53 for fraction mantissa).
- **Floating point is slow if there is no hardware support (must be emulated in software)**
- The IEEE standard for floating point arithmetic (IEEE 754) is a technical standard for floating-point computation widely implemented in hardware.

Summary: Tips

Energy Optimization

- Use registers efficiently.
- Identify and eliminate cache conflicts.
 - If your MCU has cache
- Moderate loop unrolling eliminates some loop overhead instructions.
- Eliminate pipeline stalls.
- Inlining procedures may help: reduces linkage, but may increase cache thrashing.

Efficient Loops

- Don't use function calls.
- Keep loop body small to enable local repeat (only forward branches).
- Use unsigned integer for loop counter.
- Use `<=` to test loop counter.
- Make use of compiler---global optimization

Program Size

- Reduces hardware cost of memory;
 - reduce power consumption of memory units.
- Data:
 - Reuse constants, variables, data buffers in different parts of code.
 - Generate data using instructions rather than using values.
- Avoid function inlining.
- Choose CPU with compact instructions.
 - Use specialized instructions where possible.